

Lahore University of Management Sciences

AI651 — Deep Learning for Space, Time and Graphs

Assignment 2

Name: Rifa Salman

Roll number: 25280059

Fall 2026

GitHub (public): https://github.com/RifaSalman/AI651-Assignment-2

Question 1 — River Forecasting Proposal

The forecaster uses a shared LSTM on each series for time, then self-attention across series embeddings for space (temporal-then-spatial composition). Sensors are irregular points with known river connectivity, so 2D convolution / ConvLSTM on a grid is not used. Graph message-passing layers are not used. Only information that is available at issuance τ enters the model. The product is the water level at S1–S4 at τ+60 min.

1. 1. Architecture diagram

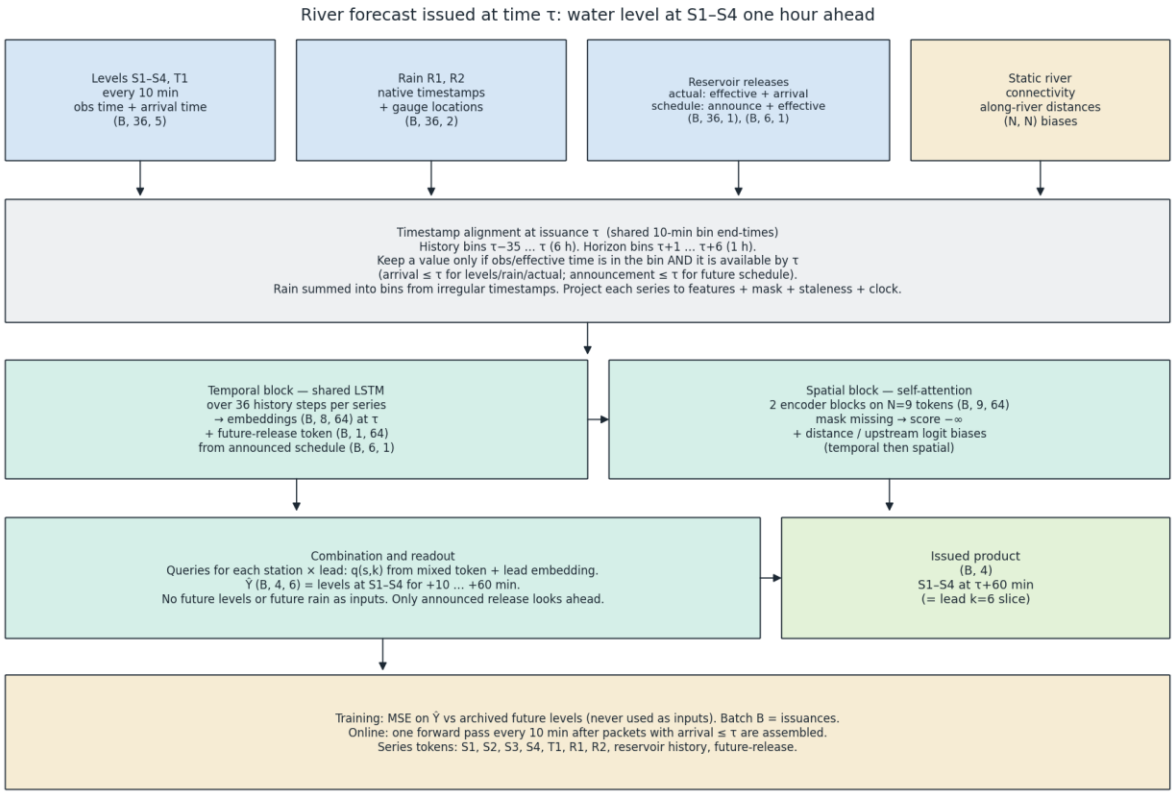


Figure Q1. Forecasting architecture. Blue boxes are observations, sand boxes are fixed by the river, green boxes are learned, and the issued product is the +60 min level at S1–S4.

Figure Q1 shows each data source, timestamp alignment, the LSTM, the attention mixer, and the S1–S4 forecasts, with main tensor shapes labelled.

Sources: levels S1–S4 and T1; rainfall R1 and R2 with native timestamps and locations; reservoir actual releases and announced schedules (announcement and effective times); static connectivity and along-river distances.

Alignment: all series map onto one 10-minute clock of bin end-times. History is τ−35 … τ. Horizon is τ+1 … τ+6. A value enters bin t only if its observation/effective time falls in that bin and it is available by τ. Each series is projected to a common feature width before the LSTM.

Blocks: shared LSTM over 36 steps per series; two self-attention blocks over the series tokens, with additive biases from distances and upstream connectivity; station-and-lead readout. Output $\hat{Y}$ is (B, 4, 6); the published slice is (B, 4) at lead +60 min.

2. Assumptions

Supplied facts: levels at S1–S4 and T1 every ten minutes; rainfall at R1 and R2 with timestamps and locations; recorded releases and future schedules announced before issuance, with announcement and effective times; station locations, connectivity and distances; no precise travel times; no unmonitored-tributary measurement; no future rainfall or future levels at issuance. The figure fixes where the tributaries join.

A1. History length. Six hours (36 bins) covers dam→S4 travel with margin relative to Assignment 1's two-hour station-3→4 example. Shorter history would hide older releases. Sets LSTM length to 36.

A2. Shared 10-minute clock. Levels are already on that grid. Rain timestamps are summed into (t−10 min, t]; a bin with no reading is 0 with rain-mask 0 (not a carried rate). Release effective times use the same bins.

A3. Observation vs arrival. Levels, rain and actual releases need observation/effective time ≤ τ and arrival time ≤ τ. Late packets are dropped, not waited on.

A4. Record length. At least one year of archived issuances. Sin/cos of hour and day-of-year are inputs. No separate seasonal model.

A5. Measurement quality and scaling. Stage is in one datum, metres, with centimetre-scale noise; there is no bias-correction block. Each series is standardized with its training mean and standard deviation and inverted on the issued forecast, so stage, rain and release can share one network width.

A6. Travel time. Not a fixed delay and not distance divided by a constant speed. Delayed effects must appear in the LSTM history. Along-river distances bias attention among the reservoir, S1–S4 and T1; gauge-to-station distance for R1/R2 is straight-line from the given coordinates. Neither distance is turned into a travel time.

A7. Rain catchments. R1 informs the unmonitored headwaters; R2 the lower valley toward S3/S4. Bias reduces R1→S1 weight.

A8. No future rain; point forecast. Future rain is not invented. The training loss is mean squared error on level (missing targets dropped). No quantile head and no separate physical simulator.

A9. Announced schedules. A release is treated as piecewise constant between effective times. A future bin is usable only if announcement time ≤ τ and effective time falls in τ+1 … τ+6. When training on the archive, each past issuance is rebuilt with only announcements that were public at that issuance.

A10. Shared model. One LSTM stack and one attention stack for all four stations.

A11. Published product. Train leads +10 … +60 min; publish only +60 min.

A12. Residual S3−S2. Kept when both masks are 1, as a cue for the unmonitored branch, not a measured inflow.

A13. Backwater. Treated as negligible; biases favour upstream→downstream influence.

3. Forecasting task and data specification

Issuance time τ is the end of a 10-minute bin. History: τ−35 … τ. Horizon: τ+1 … τ+6 (one hour). Outputs: levels at S1–S4 for each lead; service product is the four levels at τ+6.

Available at τ: levels/rain with obs and arrival ≤ τ; actual releases with effective and arrival ≤ τ; schedule rows with announcement ≤ τ and effective time in (τ, τ+60 min]; static layout. Not available: future rain, future levels, unmonitored inflow, schedules announced after τ, unarrived packets.

Alignment: (1) shared bin end-times; (2) levels into their native bin; (3) rain summed by timestamp into bins (empty ⇒ mask 0); (4) actual release by effective time, kept if arrival ≤ τ; (5) schedule to past bins or future leads by effective time, gated by announcement; (6) no spatial raster—series stay separate tokens; (7) batch B indexes issuances.

Series / tensor	Shape	Axes	Timestamps / time coverage
Level history, S1–S4 and T1	(B, 36, 5)	batch, history bin, station	bin end-times τ−35 … τ
Level mask	(B, 36, 5)	same	1 iff obs ≤ τ and arrival ≤ τ
Level staleness (minutes)	(B, 36, 5)	same	since last arrived obs
Rain history, R1 and R2	(B, 36, 2)	batch, history bin, gauge	mm from timestamps in each bin
Rain mask	(B, 36, 2)	same	1 iff a reading in the bin arrived by τ
Actual release	(B, 36, 1)	batch, history bin, reservoir	by effective time; arrival ≤ τ
Schedule in force (past)	(B, 36, 1)	same	announcement ≤ bin end
Residual S3−S2	(B, 36, 1)	batch, history bin	masked if either station missing
Announced future schedule	(B, 6, 1)	batch, lead, reservoir	effective in lead bin; announcement ≤ τ
Future-schedule known-mask	(B, 6, 1)	same	1 if a qualifying announcement exists
Distances / upstream flags	(N, N)	series pair	static
Clock features	(B, 36, 4)	batch, history bin, features	at each history bin end-time
LSTM embeddings	(B, 8, 64)	batch, series, hidden	summary at τ
Future-release token	(B, 1, 64)	batch, token, hidden	from schedule + known-mask
Mixed tokens	(B, 9, 64)	batch, token, hidden	after two attention blocks
Forecast $\hat{Y}$	(B, 4, 6)	batch, station, lead	targets τ+10 … τ+60 min
Issued forecast	(B, 4)	batch, station	τ+60 min

4. Model computations and design justification

Temporal: for series i and bin t, $u_{i,t} = [x_{i,t} \odot m_{i,t}, m_{i,t}, s_{i,t}, c_t]$,

$h_{i,t} = \mathrm{LSTM}(h_{i,t-1}, W_{type}\, u_{i,t})$, $e_i = h_{i,\tau}$,

shared by series type. Future schedule r with known-mask κ → $e_{rel} = \mathrm{MLP}([r \odot \kappa; \kappa])$. Token list

$Z^0 = [e_{S1}, e_{S2}, e_{S3}, e_{S4}, e_{T1}, e_{R1}, e_{R2}, e_{res}, e_{rel}]$.

Spatial: two transformer encoder blocks (4 heads, width 64, FFN 4d). Missing series get score $-\infty$. Logit bias $b_{ij} = \lambda_{dist} \cdot \phi(\mathrm{distance}_{ij}) + \lambda_{up} \cdot 1[i \text{ upstream of } j]$.

Readout: $q_{s,k} = W_q [Z^2_s; \mathrm{lead}_k; \mathrm{staleness}_s]$, $\hat{y}_{s,\tau+k} = w^T \mathrm{Attn}(q_{s,k}, Z^2) + b$.

Objective: MSE over stations and leads; drop missing future targets. Future levels are never inputs.

Fixed: series list, timestamp rules, connectivity, distances, unannounced ⇒invisible. Learned: LSTM, attention, biases, readout. No mass-conservation constraint.

Drivers. S1: own history plus reservoir actual/schedule (e_{res}, e_{rel}); rain only weakly via A7. S2: S1 and T1 as upstream tokens, then the reservoir with a longer lag. S3: S2 stem, rain, residual S3−S2 for the unmonitored branch, and the reservoir. S4: same upstream set through attention in one hop, with S3 as the nearest level cue. Inflow that has not reached S3 by τ remains invisible.

5. Operation and exceptional conditions

(i) Missing: value 0, mask 0, staleness updated; no interpolation; fully missing series masked in attention.

(ii) Delayed: arrival > τ ⇒drop for this issuance; later issuances may use it; no revision of issued forecasts.

(iii) Announced future releases: include in lead k only if announcement ≤ τ and effective time in that lead bin; training rebuilds each past issuance with only schedules known at that τ.

(iv) Schedule vs actual: both on past bins when available; future leads use announced schedule only.

Feasibility: length-36 LSTM on eight series, two attention blocks on nine tokens, 24 queries—well under ten minutes on CPU. Budget is packet arrival, not compute.

6. Uncertainty, limitations, and additional information

Unmonitored tributary. Impact: under-forecast until S3 rises or rain correlates. Response: residual + R1. Request: confluence gauge. Usable without it: yes.

Changing travel times. Impact: mis-timed waves at unseen stage. Response: data-driven LSTM/attention, no hard delay. Request: travel-time checks at low/high stage. Usable without them: yes.

No future rainfall. Impact: post-τ storms missed. Request: one-hour rain forecast token. Usable without it: yes.

Schedule vs actual. Impact: unannounced gate change invisible until actual arrives. Request: faster gate telemetry. Usable without it: yes if operators follow the schedule.

Datum/rating. Request: calibration dates / rating curves. Usable without them: yes (forecast reported stage).

Backwater. Request: control-structure notes. Usable without them: yes; error local.

Clock skew on arrival stamps. Request: authenticated centre clock. Usable with conservative A3: yes.

Question 2 — Motion or Memory?

Hardware: NVIDIA Tesla T4 (Kaggle). Training time is mean wall-clock seconds per epoch over 30 epochs (dataset download excluded). Inference time is wall-clock time for all 1,000 original test videos at batch 32, after each batch is on device, under eval() and no_grad(). Learned entries are mean (seed 0, seed 1). Figures and GIFs use seed 0.

Table 1. Transformation and window checks

Check	Expected	Result
(a) max |M(M(x)) − x|	0	0, torch.equal True
(b) max |R^4(x) − x|	0	0, torch.equal True
(c) max |M(R(M(x))) − R^{−1}(x)|	0	0, torch.equal True
(d) bright pixel of M(p)	(7, 10, 43)	(7, 10, 43)
(e) bright pixel of R(p)	(7, 43, 10)	(7, 43, 10)
(f) 1× input / target	1–5 / 6–10	1–5 / 6–10
(g) 2× input / target	1, 3, 5, 7, 9 / 11, 13, 15, 17, 19	same
(h) 3× input / target	1, 4, 7, 10, 13 / 16, 19	same

Table 1. Checks (a)–(h) on the functions used in the experiments. x is torch.rand(20, 1, 64, 64) after torch.manual_seed(0). p is zero except p[7, 0, 10, 20] = 1. Model A has 480,784 parameters.

Figure 1. Optical flow on the first 50 training videos (103050 pixels kept, median speed 3.32 px/frame).

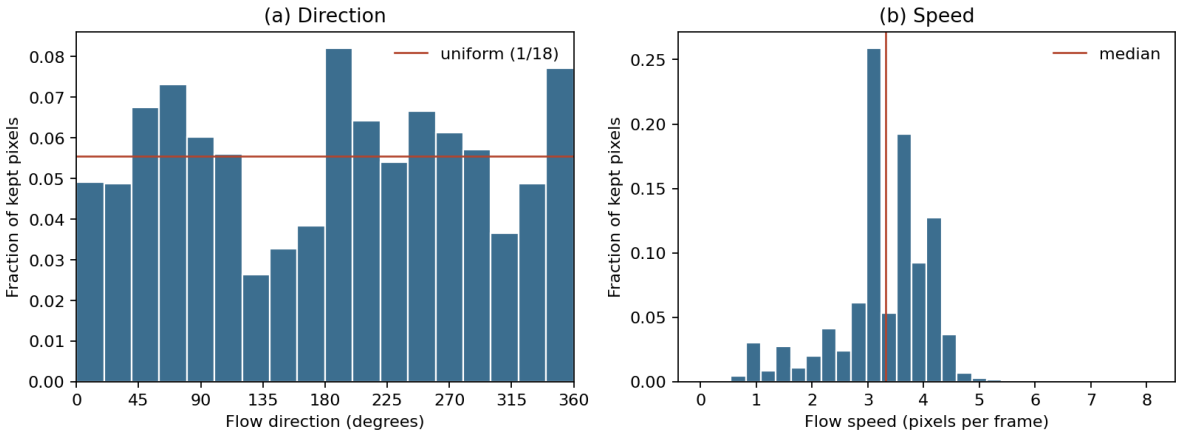


Figure 1. Optical flow on the first 50 training videos, 950 frame pairs, Farneback settings as specified. Pixels kept: intensity above 0.2 on both frames and magnitude above 0.5. Kept pixels: 103050. Median speed 3.32 pixels per frame. The horizontal line in (a) is 1/18.

Q2.1

Mirrored and rotated trajectories are partly familiar, and they are not exchangeable with the training distribution. Every 20° bin in Figure 1(a) is occupied, so a reflected or rotated direction is not a new kind of motion. The bins are not flat: the fullest holds 0.082 of the kept pixels and the emptiest 0.026, a ratio of 3.11, against a uniform level of 1/18 ≈ 0.0556. Reflecting every direction moves about 0.13 of the probability mass (total variation between the histogram and its mirror). A 90° rotation is not aligned to the 20° bins; the closest shifts, 4 and 5 bins, move about 0.18 of the mass. A typical speed is the median, 3.32 pixels per frame (mean 3.27, 90th percentile 4.12). Twice that median is 6.63 pixels per frame. Three times it is 9.95. Twice the 90th percentile is 8.25. Optical flow is unreliable where digits overlap. The kept set already drops pixels that are dark in either frame, but overlap boundaries can still invent vectors and fatten the tails. That bias would have to move the median a long way to change the comparison with an 8-pixel reach. It does not: even a median of 4 would put 2× on the reach and 3× beyond it.

Q2.2

A mirror changes both the appearance and the motion statistics. A mirrored digit is a different shape, and the direction histogram is reflected. A 90° rotation changes appearance more than a mirror does, and it sends every direction into a different part of Figure 1(a). A speed-up does not change the digit. It only increases the displacement between the frames the model sees. The three therefore separate appearance, an unfamiliar direction, and the size of the step. Constant-velocity motion with elastic bounces is still valid motion after all three, so a model that had learned the motion rule should cope with all of them.

Table 2. Baseline forecast error

Model	MSE1	MSE2	MSE3	MSE4	MSE5	E(5)	Parameters
A	0.0171 (0.0173, 0.0168)	0.0205 (0.0209, 0.0201)	0.0228 (0.0233, 0.0223)	0.0246 (0.0251, 0.0241)	0.0263 (0.0268, 0.0259)	0.0223 (0.0227, 0.0218)	480,784
Blank	0.0426	0.0426	0.0426	0.0425	0.0425	0.0426	0
Copy-last	0.0492	0.0604	0.0669	0.0708	0.0732	0.0641	0

Table 2. Test-set MSE by horizon at 1×, t = 1. Learned entries are mean (seed 0, seed 1).

Figure 2. Seed-0 model A on three test videos. Rows are input, prediction, and ground truth.



Figure 2. Seed-0 model A on test videos 9000, 9007 and 9012. Each block is input, prediction, ground truth, five frames each. Video 9000 is the overlapping pair (a 5 and a 6 merge and separate). Videos 9007 and 9012 keep two components through frames 1–10.

Q2.3

pixel_unshuffle(4) makes each cell of the 16×16 grid stand for 4 original pixels, centre to centre. The 4×4 patch width is not added. The 1×1 head, the unshuffle and the shuffle do not move information. Each ConvLSTM layer is a 3×3 convolution with dilation 1, so it reads offsets −1, 0 and +1 and adds a Chebyshev radius of one cell. In one step the new input passes through layer 1 and then layer 2. Layer 1's previous hidden state takes that same path. Layer 2's previous state passes through layer 2 only, which is shorter. The longest path is 2 cells, 8 original pixels along a row or a column. The corner of that neighbourhood is 8√2 ≈ 11.3 pixels. The reach reported here is the axis-aligned one, 8 pixels.

Q2.4

Model A's E(5) is 0.0223, against 0.0426 for blank and 0.0641 for copy-last. It is clearly better than both references on ordinary 1× clips. The error grows with the horizon, from MSE1 = 0.0171 to MSE5 = 0.0263. That is the closed-loop rollout: each later frame is conditioned on the network's own previous prediction, so blur accumulates. In Figure 2 the separated digits in videos 9007 and 9012 stay recognisable and follow the right general direction, while becoming softer. Video 9000 is the overlap. The 5 and the 6 merge, and the forecast turns them into one soft blob instead of recovering two digits the way the ground truth does. Overlap is where the model loses which digit is which, not only where it is.

Q2.5 (written before any mirror, rotation, or speed experiment)

Model A will fail the mirror test and the rotation test. It is trained only on ordinary 1× clips, and a plain ConvLSTM has no mirror or 90° symmetry built in. Figure 1(a) is not flat: every 20° bin is occupied, so a reflected or rotated direction is not a completely new kind of motion, but the fullest bin is about 3.1 times the emptiest. Mirroring the directions moves about 13% of the probability mass, and a 90° rotation moves about 18%. I expect the network to tie its forecast to the digit shapes and to the directions it saw most often. Rotation should be worse than mirroring, because a 90° turn changes what a digit looks like more, and it sends every direction into a different part of the histogram.

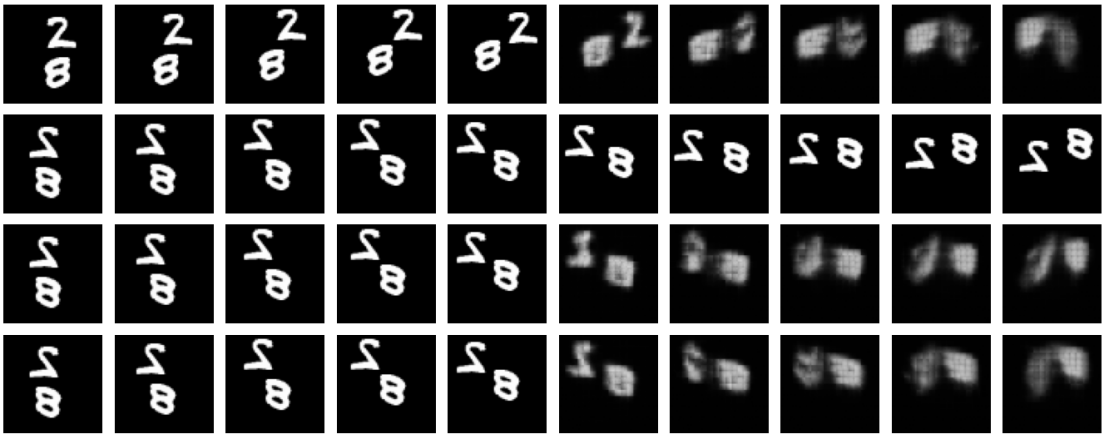
On speed, model A should mostly keep up at 2× and break at 3×. One step reaches 8 original pixels: the input passes through two 3×3 convolutions on the 16×16 grid, each grid step is 4 original pixels, and the 4×4 patch width is not added. Figure 1(b) has median speed 3.32 pixels per frame and a 90th percentile of 4.12. Twice the median is 6.64, still inside 8 pixels. Three times the median is 9.95, which is beyond an axis-aligned step of 8. The fast tail at 2× is about 8.2 pixels per frame, so a few 2× clips may already smear, but the clear breakdown should be at 3×.

Table 3. Mirror and rotation

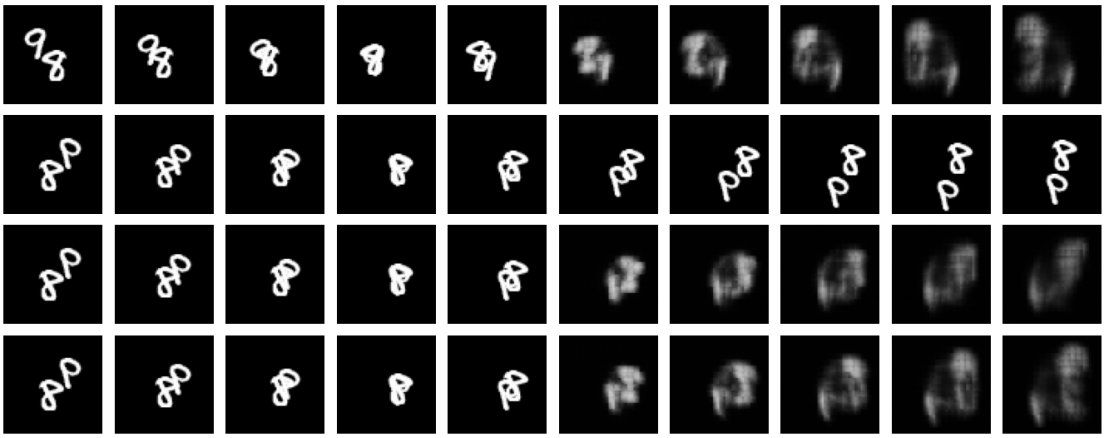
Model	E(5)	C_M	E_M	C_R	E_R
A	0.0223 (0.0227, 0.0218)	0.00174 (0.00145, 0.00203)	0.0229 (0.0232, 0.0225)	0.00293 (0.00237, 0.00349)	0.0239 (0.0241, 0.0238)
Blank	0.0426	0	0.0426	0	0.0426
Copy-last	0.0641	0	0.0641	0	0.0641

Table 3. Consistency gaps and transformed error at 1×, t = 1. Blank and copy-last are exactly consistent. Their C_M and C_R are 0.

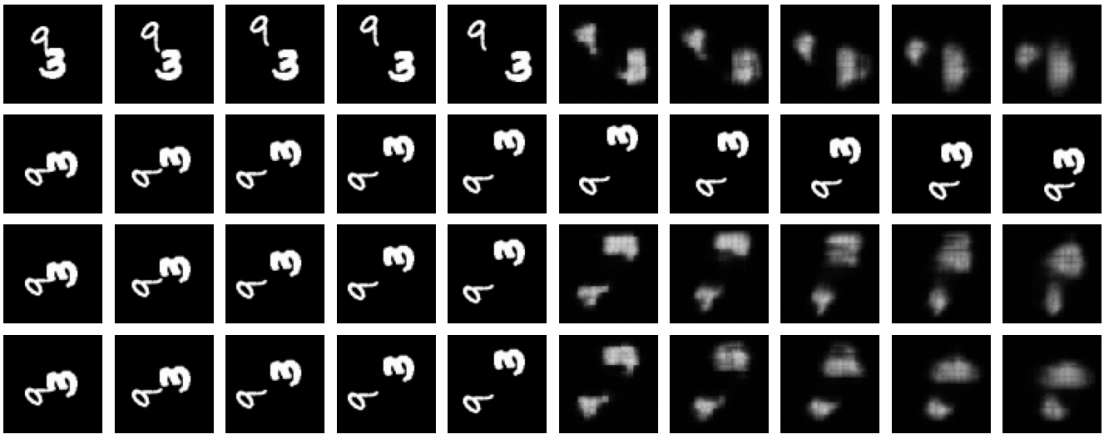
mirror video 9642



mirror video 9279



rotation video 9168



rotation video 9925

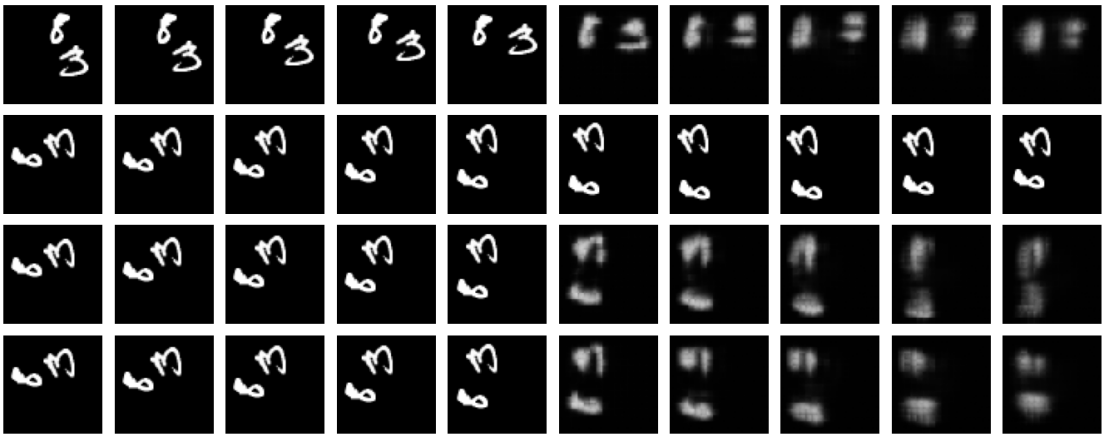


Figure 3. Seed-0 model A, the two test videos with the largest per-video C_M (9642, 9279) and the two with the largest per-video C_R (9168, 9925). Rows in each panel are X | f(X); T(X) | T(Y); T(X) | f(T(X)); T(X) | T(f(X)). GIFs: Question2/outputs/gifs/figure3_mirror_0.gif, figure3_mirror_1.gif, figure3_rotation_0.gif, figure3_rotation_1.gif.

Figure 4. Seed-0 model A consistency gaps on the test set. Marked videos are those in Figure 3.

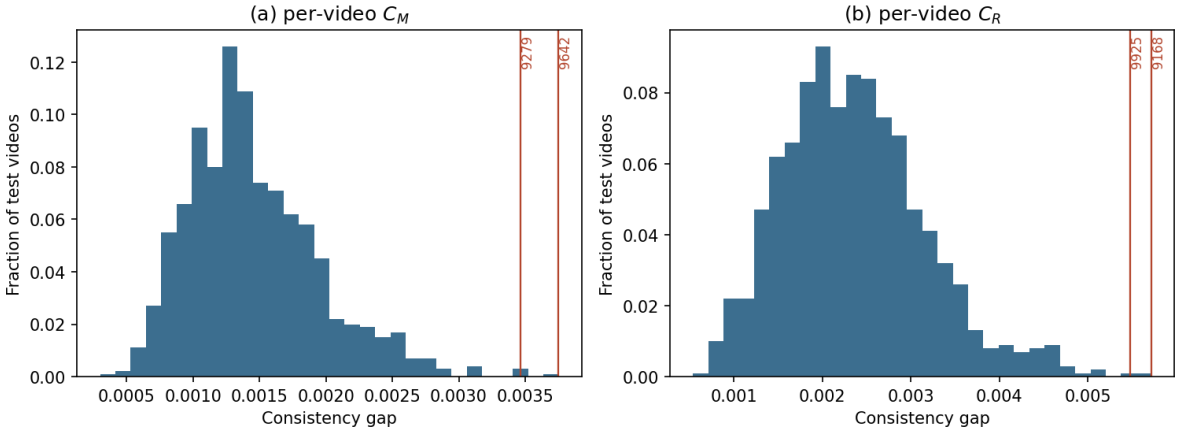


Figure 4. Per-video C_M and C_R of seed-0 model A on the 1,000 test videos. Red lines mark the videos in Figure 3.

Q2.6

C_T = 0 means the forecast commutes with T. It does not mean the forecast is right. Blank and copy-last commute for a structural reason: a black frame is still black after a mirror or a rotation, and repeating the last frame commutes with a pixel permutation. Their transformed error equals their original error, 0.0426 and 0.0641, so they are consistent and wrong. E_T has to sit next to C_T. Otherwise a constant predictor looks perfect.

Q2.7

Video 9168, the largest rotation gap for seed 0 (C_R = 0.00572). Rows 3 and 4 of that panel share the rotated input. Row 4 is the original forecast after the same rotation: both digits are still there, blurred, and they keep moving apart. Row 3 is a fresh forecast from the rotated input: one digit fades within the five horizons and the other smears into a block. The model's account of the motion changes. Rotating a forecast it was willing to make on the upright clip does not match the forecast it makes once the clip is rotated. Shown the rotated clip, it loses a digit instead of continuing both trajectories.

Q2.8

Yes. Mean C_R is 0.00293 against mean C_M of 0.00174, and E_R is 0.0239 against E_M of 0.0229 and an original E(5) of 0.0223. Figure 4 sits further to the right for rotation than for mirror. From Q2.1, a 90° rotation moves more of the direction histogram than a mirror does. From Q2.2, it also changes the shape of a digit more. Both of those are about appearance and about which directions were common, not about a different law of motion. The transformed errors stay far below the blank predictor (0.0426). The model is still forecasting. The speed test, which changes the step and not the appearance, is the one that reaches the blank error. That is a different failure.

Table 4. Faster motion

Model	E(2) at 1×	E(2) at 2×	E(2) at 3×	E(5) at 1×	E(5) at 2×
A	0.0188 (0.0191, 0.0184)	0.0449 (0.0437, 0.0461)	0.0523 (0.0512, 0.0535)	0.0223 (0.0227, 0.0218)	0.0497 (0.0474, 0.0519)

Copy-last	0.0548	0.0661	0.0703	0.0641	0.0719

Table 4. Speed comparison on the test set at t = 1. E(5) is not defined at 3×, because only two target frames exist. Blank's E(2) is 0.0426, 0.0425 and 0.0426 at 1×, 2× and 3×.

Figure 5. Five input frames, the first two predictions, and the first two true futures at 1×, 2× and 3×.

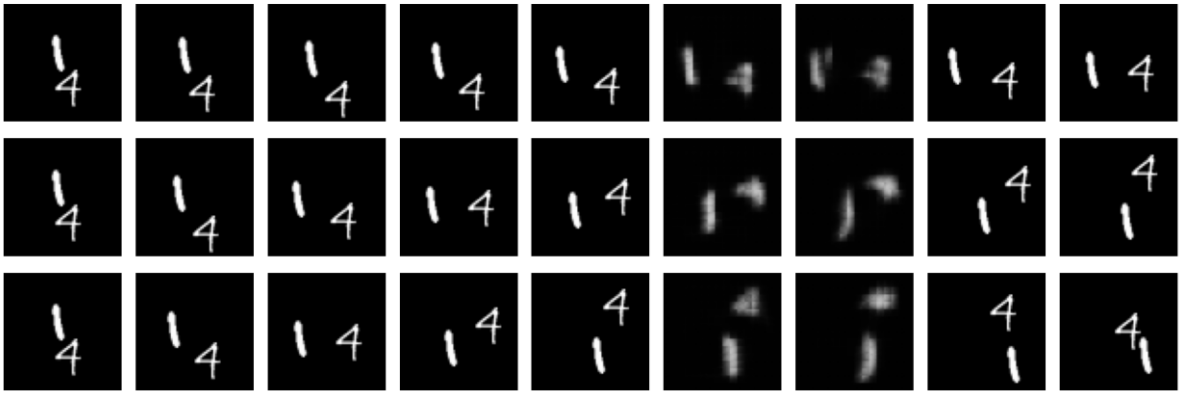


Figure 5. Seed-0 model A on test video 9021, chosen so the displayed true frames do not overlap. Each row is one speed: five input frames, the first two predictions, and the first two true future frames, at 1×, 2× and 3×.

Q2.9

Model A falls apart at 2×, and 3× is worse. E(2) goes from 0.0188 at 1× to 0.0449 at 2× and 0.0523 at 3×. Blank's E(2) is about 0.0426, so at 2× the forecast is already no better than predicting black. This is earlier than the median argument in Q2.5. Twice the median speed is 6.63 pixels, inside a reach of 8, and three times the median is 9.95, outside it. What that estimate missed is the tail and the width of a digit. Twice the 90th percentile is 8.25 pixels, already past the axis-aligned reach, so a real share of 2× motion cannot be joined to the previous position in one update. A digit is also wider than the one pixel whose flow was measured. Figure 5, video 9021, shows the failure as smear. At 1× the 1 and the 4 stay sharp enough to follow. At 2× and 3× the forecast is a soft ghost near the last input position, while the true digits have moved on. A stall would repeat a sharp digit. A duplicate would show two copies. A vanishing digit would go to black. This is the smear: the model no longer has a position to commit to.

Q2.10

No. Model A never trained on k = 2 or k = 3, and 2× motion also sits on the edge of the 8-pixel reach. Either fact can produce Figure 5. The split is model C, the same architecture trained with k drawn from {1, 2, 3}, against model D, a larger reach trained on the same speeds. If C repairs 2× and still fails at 3×, then 2× was missing from the training distribution and 3× is beyond the architecture. If C fails at 2× as well, the reach was already the limit, and more fast clips cannot move information further than the convolutions allow.

Q2.11 (written before training models B, C, and D)

Model B should remove the mirror and rotation failures. Those errors stay well below the blank predictor: transformed E(5) is only a little higher than the original E(5), so this looks like a missing symmetry rather than a broken motion rule, and B is the only one of these three models that sees g drawn from G. Model C should be the one that repairs 2×, because twice the median speed is 6.6 pixels, inside the 8-pixel reach, and C is the first model that actually sees k = 2; it should not repair 3×, where a step of about 10 pixels is beyond that reach and E(2) is already worse than blank. Model D keeps C's speed augmentation and extends the reach to

16 pixels, so it is the model that should bring 3× back, while the mirror and rotation gaps stay until the forecaster is symmetrised.

Q2.12

Model D is model A's ConvLSTM with one change. Layer 1 stays a 3×3 with dilation 1. Layer 2 is a 3×3 with dilation 3 and padding 3, so it reads offsets −3, 0 and +3. Dilation adds no weights, so the parameter count stays 480,784, under the limit of 721,176. There is no attention and no extra layer. The 9-step rollout, the shapes, the loss and the rest of the training protocol are unchanged. The only protocol change is the one in the model table: k is drawn from {1, 2, 3}. Reach uses the same rules as Q2.3. One cell is 4 original pixels. Layer 1 adds a radius of 1 cell and layer 2 adds a radius of 3. The longest path, the new input or layer 1's previous state through both convolutions, has Chebyshev radius 4 cells, which is 16 original pixels. The summed offsets are every integer from −4 to 4, so that neighbourhood has no holes. Layer 2's own previous state only reaches 12 pixels, which is the shorter path. 16 is strictly larger than model A's 8. Three times the median speed, 9.95, and three times the 90th percentile, about 12.4, are inside 16 and outside 8.

Table 5. What changes with the data, and with the reach

Model	E(5)	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	Params	Train s	Infer s
A	0.0223 (0.0227, 0.0218)	0.00174 (0.00145, 0.00203)	0.0229 (0.0232, 0.0225)	0.00293 (0.00237, 0.00349)	0.0239 (0.0241, 0.0238)	0.0188 (0.0191, 0.0184)	0.0449 (0.0437, 0.0461)	0.0523 (0.0512, 0.0535)	480,784	11.9 (11.9, 11.9)	0.42 (0.421, 0.42)
B	0.0228 (0.0236, 0.0221)	9.82e-4 (0.00101, 9.58e-4)	0.0228 (0.0236, 0.0221)	0.00125 (0.00125, 0.00125)	0.0228 (0.0235, 0.0221)	0.0191 (0.0196, 0.0186)	0.0425 (0.0419, 0.0432)	0.048 (0.0467, 0.0493)	480,784	12.5 (12.5, 12.5)	0.422 (0.423, 0.422)
C	0.027 (0.0278, 0.0262)	0.00136 (0.00145, 0.00127)	0.0272 (0.0279, 0.0265)	0.0026 (0.0022, 0.00299)	0.0282 (0.0287, 0.0276)	0.0233 (0.0241, 0.0224)	0.0288 (0.0295, 0.0281)	0.0336 (0.0339, 0.0332)	480,784	11.8 (11.8, 11.8)	0.423 (0.422, 0.424)
D	0.0305 (0.0311, 0.0299)	3.36e-4 (2.45e-4, 4.28e-4)	0.0305 (0.0311, 0.0299)	7.90e-4 (6.45e-4, 9.35e-4)	0.0308 (0.0314, 0.0303)	0.028 (0.0285, 0.0275)	0.0311 (0.0316, 0.0307)	0.0342 (0.0345, 0.034)	480,784	16.1 (16.1, 16.1)	0.613 (0.614, 0.613)
D+B	0.0302 (0.0308, 0.0295)	2.44e-4 (1.51e-4, 3.38e-4)	0.0302 (0.0308, 0.0296)	2.90e-4 (1.95e-4, 3.86e-4)	0.0302 (0.0308, 0.0296)	0.0277 (0.0282, 0.0273)	0.0311 (0.0315, 0.0307)	0.034 (0.0344, 0.0336)	480,784	16.6 (16.6, 16.6)	0.613 (0.613, 0.612)

Table 5. Test metrics for the learned models. D+B is model D trained with both speed and symmetry augmentation. Train s is the mean seconds per epoch. Infer s is one forward pass of the 1,000 test videos after the batch is on the Tesla T4.

Q2.13

The mirror and rotation gaps are an appearance failure, in the sense of Q2.2: the motion law is unchanged and the digits are not. Model B, which only adds g drawn from G, cuts C_M from 0.00174 (0.00145, 0.00203) to 9.82e-4 (0.00101, 9.58e-4) and C_R from 0.00293 (0.00237, 0.00349) to 0.00125 (0.00125, 0.00125). Its transformed errors fall into line with its own E(5) of 0.0228 (0.0236, 0.0221). Model C, which changes speed and not symmetry, leaves C_R at 0.0026 (0.0022, 0.00299), close to model A. So the symmetry gap moves when the training distribution is symmetrised, and it does not move when only the speeds change. The speed failure is unfamiliar motion, not architectural reach. Model C has the same 8-pixel reach as A and cuts E(2) at 2× from 0.0449 (0.0437, 0.0461) to 0.0288 (0.0295, 0.0281) and at 3× from 0.0523 (0.0512, 0.0535) to 0.0336 (0.0339, 0.0332). Both of those are better than blank, which sits near 0.0426. Model D, with reach 16 and the same speed augmentation, does not beat C: its 3× E(2) is 0.0342 (0.0345, 0.034). The conditional in the question does not hold. C is not failing at 3× while D succeeds. More examples of fast motion were enough, inside the original reach. That does not mean data can extend a reach. A 3×3 kernel has a zero weight on every pixel outside its offsets, for every value of the weights. Training changes those weights. It does not

change which pixels are read. D was the test of that limit, and on this data the limit was not what separated the errors.

Q2.14

Yes. Original E(5) goes from 0.0223 (0.0227, 0.0218) for A to 0.0228 (0.0236, 0.0221) for B, 0.027 (0.0278, 0.0262) for C, 0.0305 (0.0311, 0.0299) for D and 0.0302 (0.0308, 0.0295) for D+B. B's cost is small beside the drop in the consistency gaps, and it is worth it. C's cost is larger, and it is the cost of no longer collapsing to the blank predictor at 2× and 3×. That trade is worth it for the question being asked. D's further cost over C bought no speed improvement. The dilation was not worth it on these clips. D+B matches D on the original clips and is the better of those two under mirror and rotation, which is why it is the one wrapped below.

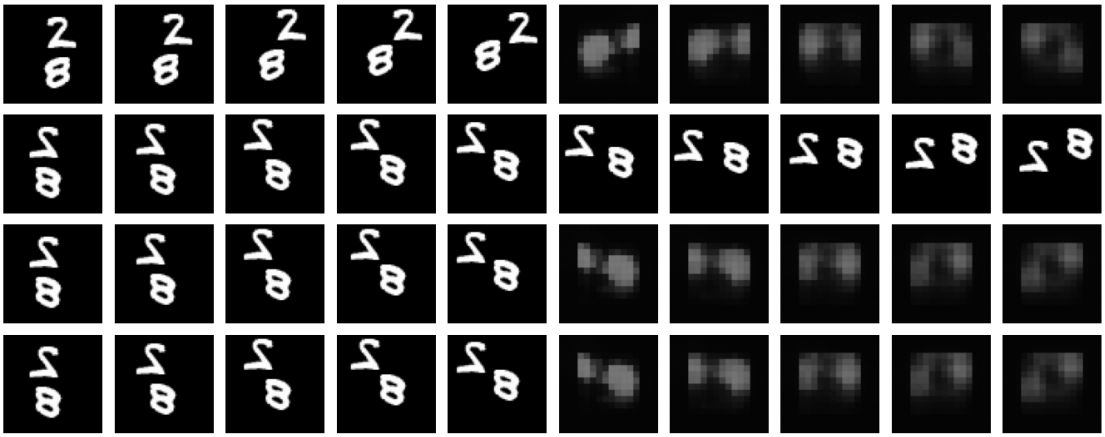
Table 6. Symmetrised forecasters

For any g in G and any T in {M, R}, right-multiplication by T is a bijection of G. Set h = g T, so g^{−1} = T h^{−1}. Then $\bar{f}$(TX) = (1/8) Σ_g g^{−1} f(g T X) = (1/8) Σ_h T(h^{−1} f(h X)) = T($\bar{f}$(X)), because T permutes pixels and therefore passes through the average. So $\bar{f}$(TX) = T($\bar{f}$(X)), and C_M = C_R = 0 for every underlying f. The values below are the floating-point remainder.

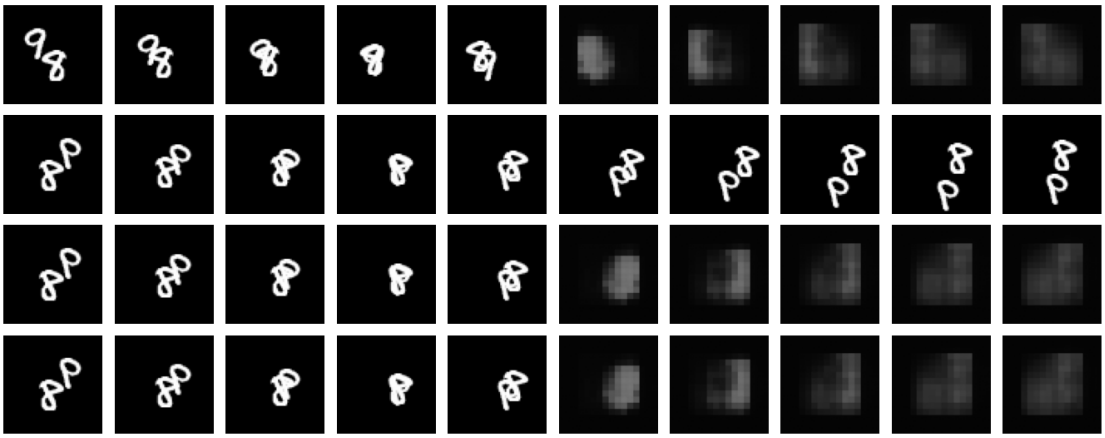
Model	E(5)	C_M	E_M	C_R	E_R	E(2) 1×	E(2) 2×	E(2) 3×	Params	Train s	Infer s
sym B	0.0223 (0.023, 0.0216)	7.21e-17 (7.09e-17, 7.33e-17)	0.0223 (0.023, 0.0216)	5.82e-17 (5.74e-17, 5.90e-17)	0.0223 (0.023, 0.0216)	0.0187 (0.0192, 0.0182)	0.0417 (0.0413, 0.0422)	0.0469 (0.046, 0.0477)	480,784	12.5 (12.5, 12.5)	3.43 (3.42, 3.44)
sym D	0.0304 (0.031, 0.0298)	5.06e-17 (5.10e-17, 5.02e-17)	0.0304 (0.031, 0.0298)	3.84e-17 (3.92e-17, 3.75e-17)	0.0304 (0.031, 0.0298)	0.0279 (0.0285, 0.0274)	0.0312 (0.0316, 0.0307)	0.0342 (0.0345, 0.0339)	480,784	16.1 (16.1, 16.1)	4.93 (4.92, 4.94)
sym D+B (final)	0.0301 (0.0308, 0.0294)	4.37e-17 (3.81e-17, 4.93e-17)	0.0301 (0.0308, 0.0294)	3.31e-17 (2.89e-17, 3.72e-17)	0.0301 (0.0308, 0.0294)	0.0276 (0.0281, 0.0271)	0.031 (0.0315, 0.0305)	0.0339 (0.0343, 0.0335)	480,784	16.6 (16.6, 16.6)	4.94 (4.93, 4.95)

Table 6. The same columns as Table 5, for the symmetrised forecasters. Train s is the underlying learned model. Symmetrisation is not trained. C_M and C_R are floating-point zeros and are written in scientific notation. The final model is symmetrised D+B, not symmetrised D.

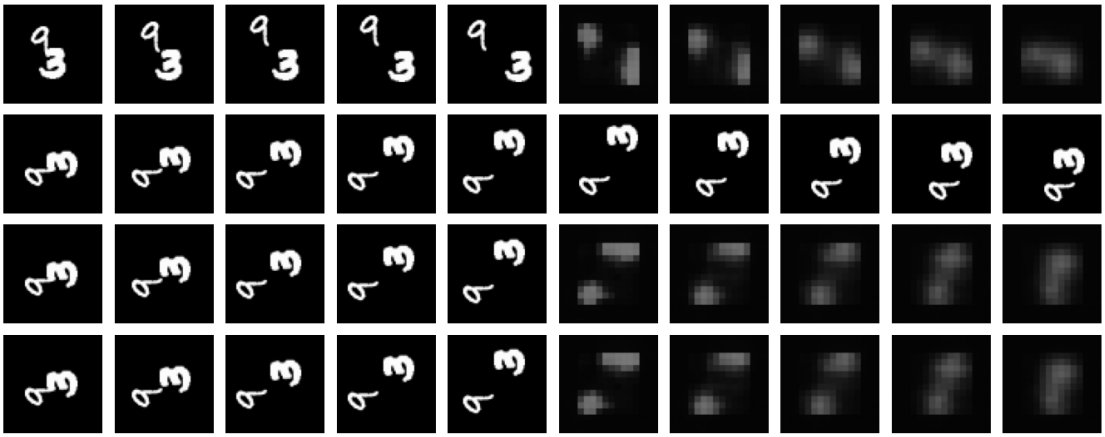
DB mirror video 9642



DB mirror video 9279



DB rotation video 9168



DB rotation video 9925

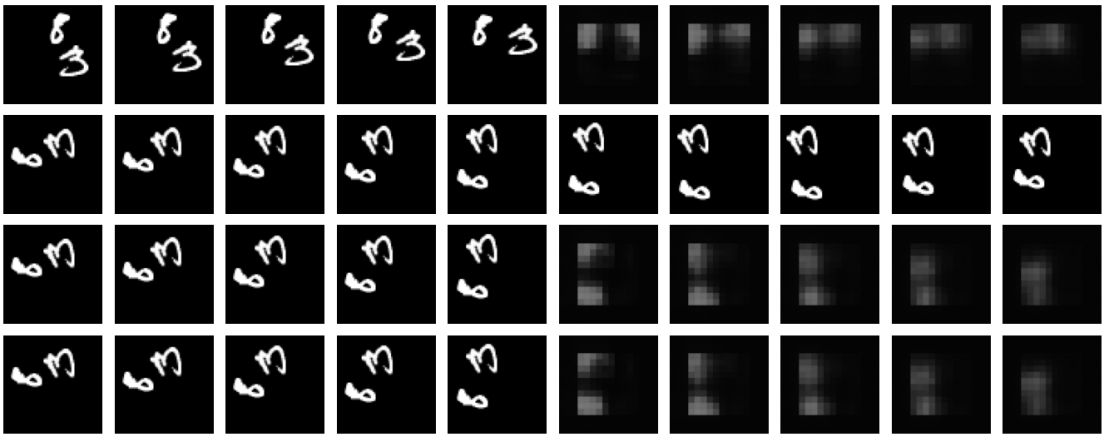


Figure 6(a). The final model, symmetrised D+B, seed 0, on the same videos as Figure 3. GIFs: Question2/outputs/gifs/figure6_DB_mirror_0.gif, figure6_DB_mirror_1.gif, figure6_DB_rotation_0.gif, figure6_DB_rotation_1.gif.

Figure 5. Five input frames, the first two predictions, and the first two true futures at 1×, 2× and 3×.

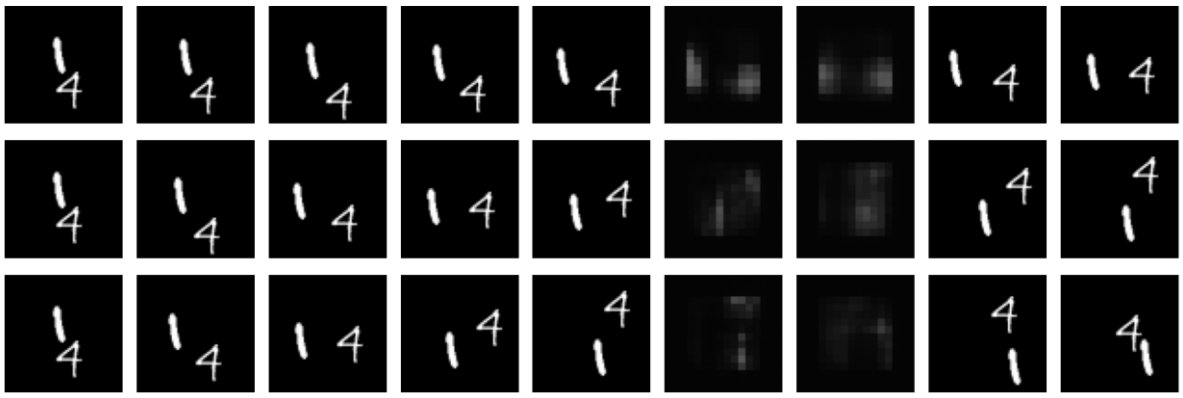


Figure 6(b). The same final model on test video 9021, the video in Figure 5, at 1×, 2× and 3×.

Q2.15

Learned symmetry is model B. Its C_M is 9.82e-4 (0.00101, 9.58e-4) and its C_R is 0.00125 (0.00125, 0.00125), against 0.00174 (0.00145, 0.00203) and 0.00293 (0.00237, 0.00349) for A. Its E_M and E_R match its E(5) of 0.0228 (0.0236, 0.0221). The gap shrinks. It does not become zero. Inference stays one forward pass, 0.422 (0.423, 0.422) s. Built-in symmetry is the average over G. It is the exact one, because the short argument above does not depend on f. Wrapped B has C_M = 7.21e-17 (7.09e-17, 7.33e-17) and C_R = 5.82e-17 (5.74e-17, 5.90e-17). The cost is the eight passes: 3.43 (3.42, 3.44) s against 0.422 (0.423, 0.422) s, and 4.94 (4.93, 4.95) s for the final model against 0.613 (0.613, 0.612) s for D+B unwrapped. Averaging eight forecasts blurs them when they disagree about where a digit is. Here they mostly agree. Wrapped B's E(5) is 0.0223 (0.023, 0.0216), no worse than learned B, and wrapped D+B's E(5) is 0.0301 (0.0308, 0.0294), the same as D+B to three figures. Built-in symmetry did not hurt accuracy on this test set. It would hurt a model whose eight transformed forecasts pointed different ways.

Q2.16

On video 9168 the disagreement is gone and the forecast is not repaired. In Figure 3, row 4 kept two blurred digits and row 3 lost one. In Figure 6(a) those two rows match: both are the same coarse, blocky field. That match is the group average. It is why C_R is a floating-point zero. It is not a better account of the bounce. The upright forecast in row 1 is blocky as well, which lines up with the original E(5) rising from 0.0223 for A to about 0.030 for D+B. The part of the failure that was a broken symmetry was fixed by the wrapper, and by D+B's symmetry augmentation before that. The part that was a lost digit was not turned back into two sharp trajectories. Figure 6(b) is the same story at speed. The error at 3× is lower than model A's, and the frames are still smears rather than a 1 and a 4 at the true positions.

Q2.17

The final model has not learned a motion rule that stands without help. Model C, with A's 8-pixel reach, cuts E(2) at 2× from 0.0449 to 0.0288 and at 3× from 0.0523 to 0.0336, so the speed collapse was motion the training clips never showed. D's 16-pixel reach does not beat C, so reach was not the binding limit. Mirror and rotation consistency are exact only because the final forecaster averages all eight symmetries. That is imposed. Model B learns a smaller gap, not a zero gap. A skeptic should ask for a change that neither the

training distribution nor the wrapper covers. The test it has not passed is a rotation by an angle other than a multiple of 90°, or a speed above 3×. Figure 6 is the other warning: the error can fall while the digits turn into blocks.

AI use

I used Cursor to help with understanding the assignment, running notebooks, writing this report, pushing code to GitHub, and polishing answers. Experiment numbers in the tables come from the saved score files. Q2.5 and Q2.11 match the prediction files committed before those experiments.

Example prompts:

Prompt 1: "Kaggle failed with ModuleNotFoundError: No module named q2 when I pushed the training script. What does that mean and how do I fix the import path so the kernel can see my package?" Cursor explained that the kernel only sees attached dataset files and helped adjust the entry script so q2 is on the path.

Prompt 2: "Model D uses dilations (1, 3) for reach 16, but its 3× E(2) is slightly worse than Model C in my table. Why would a larger reach not help here?" Cursor walked through unfamiliar motion vs reach, and that dilation 3 can skip nearby cells, so C fixing most of the gap from seeing fast clips is consistent with the results.